

BRACT's
Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: E

Batch: 1

Practical No: 2

Roll no: 52

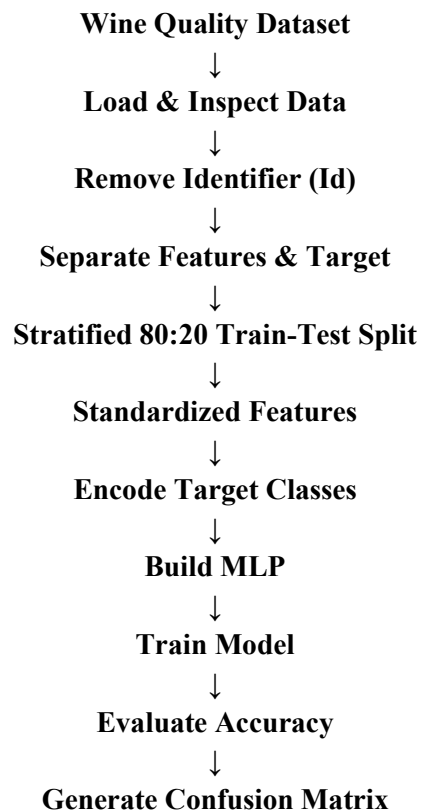
PRN No: 12410371

Name of Student: Vaidik Samarth

Subject Name: Deep Learning

Title of Practical: Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset and evaluate its performance using accuracy and a confusion matrix.

Methodology:



Code & Output:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay, classification_report

print("TensorFlow version:", tf.__version__)

--- TensorFlow version: 2.20.0
```

```
df = pd.read_csv("WineQT.csv")

print("Dataset shape:", df.shape)
display(df.head())

print("\nmissing values:", df.isnull().sum().sum())
print("\nquality distribution:")
print(df["quality"].value_counts().sort_index())
```

--- Dataset shape: (1143, 13)

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	ph	sulphates	alcohol	quality	id
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	0
1	7.8	0.88	0.00	2.6	0.096	25.0	87.0	0.9968	3.20	0.68	9.8	5	1
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	3
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	4

Missing values: 0

quality distribution:
quality
3 6
4 33

```
X = df.drop(columns=["quality", "id"])
y = df["quality"]

print("Feature matrix shape:", X.shape)
print("Target classes:", sorted(y.unique()))

--- Feature matrix shape: (1143, 11)
Target classes: [np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
```

+ Code

+ Text

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

--- Training samples: 914
Testing samples: 229
```

+ Code

+ Text

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaled training shape:", X_train_scaled.shape)
print("Scaled testing shape:", X_test_scaled.shape)

--- Scaled training shape: (914, 11)
Scaled testing shape: (229, 11)
```

```

classes = np.sort(y.unique())
class_to_index = {label: index for index, label in enumerate(classes)}

y_train_encoded = y_train.map(class_to_index).to_numpy()
y_test_encoded = y_test.map(class_to_index).to_numpy()

print("Classes:", classes)
print("Encoded classes:", np.unique(y_train_encoded))

```

Classes: [3 4 5 6 7 8]
Encoded classes: [0 1 2 3 4 5]

```

tf.random.set_seed(42)

model = tf.keras.Sequential([
    tf.keras.Input(shape=(X_train_scaled.shape[1],)),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dropout(0.20),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(len(classes), activation="softmax")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	768
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 6)	186

Total params: 2,934 (11.90 KB)
Trainable params: 2,934 (11.90 KB)

```

history = model.fit(
    X_train_scaled,
    y_train_encoded,
    epochs=50,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

```

Show hidden output

```

test_loss, test_accuracy = model.evaluate(X_test_scaled, y_test_encoded, verbose=0)

print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy:.2%}")

```

Test loss: 0.9535
Test accuracy: 64.19%

+ Code + Text

```

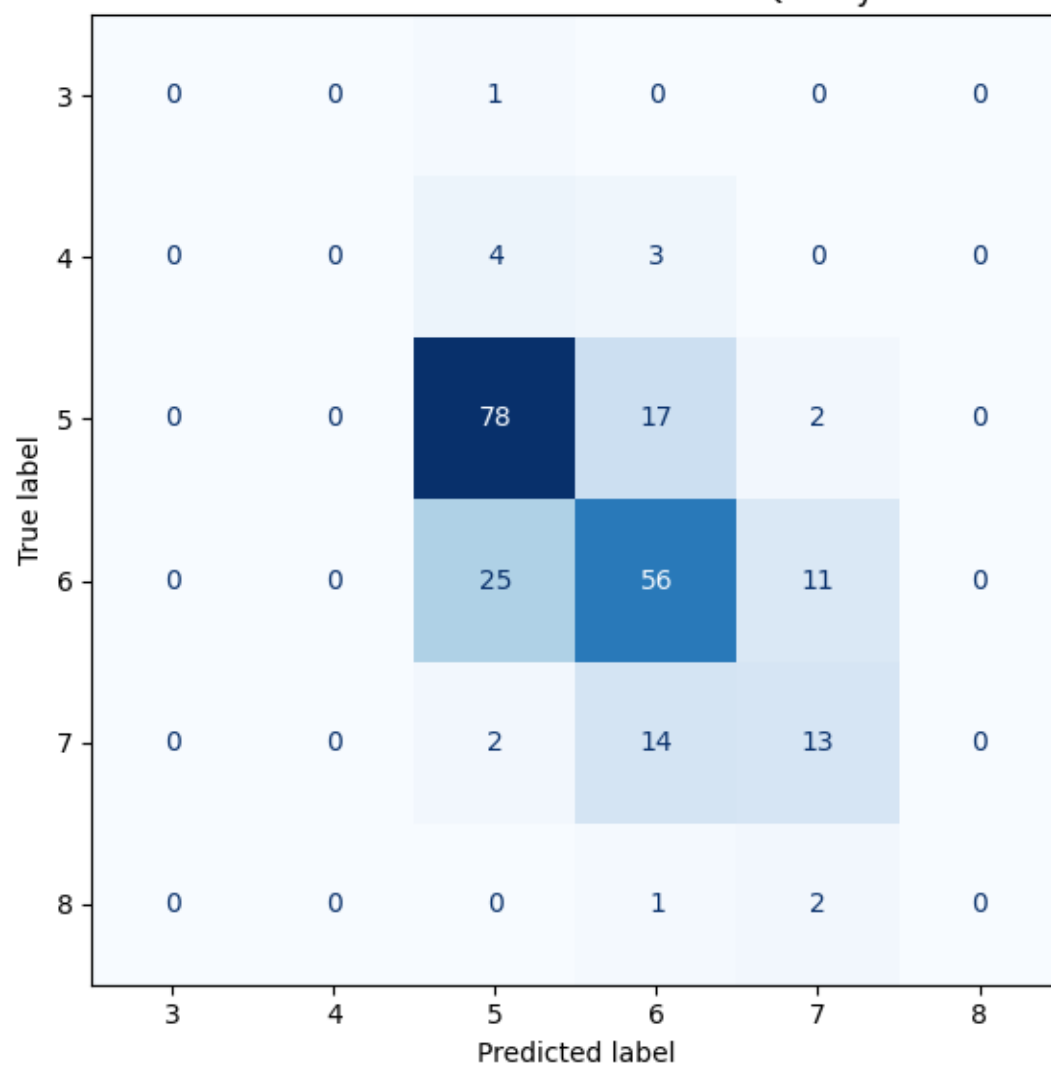
probabilities = model.predict(X_test_scaled, verbose=0)
predicted_indices = np.argmax(probabilities, axis=-1)
predicted_labels = classes[predicted_indices]

cm = confusion_matrix(y_test, predicted_labels, labels=classes)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes)
fig, ax = plt.subplots(figsize=(7, 6))
disp.plot(ax=ax, cmap="Blues", values_format="d", colorbar=False)
ax.set_title("MLP Confusion Matrix - Wine Quality")
plt.tight_layout()
plt.show()

```

MLP Confusion Matrix — Wine Quality



```
print(classification_report(
    y_test,
    predicted_labels,
    labels=classes,
    target_names=[str(c) for c in classes],
    zero_division=0
))
```

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	7
5	0.71	0.80	0.75	97
6	0.62	0.61	0.61	92
7	0.46	0.45	0.46	29
8	0.00	0.00	0.00	3
accuracy			0.64	229
macro avg	0.30	0.31	0.30	229
weighted avg	0.61	0.64	0.62	229

```

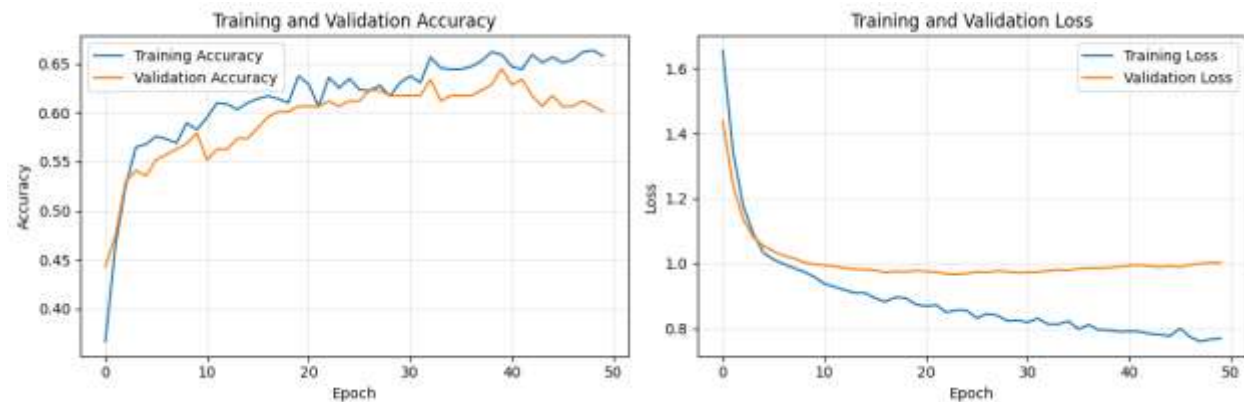
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history["accuracy"], label="Training Accuracy")
axes[0].plot(history.history["val_accuracy"], label="Validation Accuracy")
axes[0].set_title("Training and Validation Accuracy")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(history.history["loss"], label="Training loss")
axes[1].plot(history.history["val_loss"], label="Validation loss")
axes[1].set_title("Training and Validation loss")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



Conclusion:

The Wine Quality dataset was prepared using a stratified train-test split and feature standardization. A Multilayer Perceptron was then trained for multiclass wine-quality classification. Model performance was evaluated using test accuracy and a confusion matrix.